

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

C:\Users\Samruddhi Gunjal\anaconda3\Lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.3.6' or newer of 'bo
ttleneck' (version '1.3.5' currently installed).
  from pandas.core import (
```

```
In [2]: (X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

print(X_train.shape)    # (60000, 28, 28)
print(X_test.shape)     # (10000, 28, 28)

(60000, 28, 28)
(10000, 28, 28)
```

```
In [3]: # Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape (CNN needs 4D input)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# One-hot encoding
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

```
In [4]: model = Sequential()

# Convolution Layer
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))

# Pooling Layer
model.add(MaxPooling2D(pool_size=(2,2)))

# Second Convolution
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# Flatten
model.add(Flatten())

# Fully Connected Layer
model.add(Dense(128, activation='relu'))

# Output Layer (10 classes)
model.add(Dense(10, activation='softmax'))

C:\Users\Samruddhi Gunjal\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input_shape`/`i
nput_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [5]: model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
In [6]: history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2
)

Epoch 1/5
375/375 ----- 35s 85ms/step - accuracy: 0.7044 - loss: 0.8584 - val_accuracy: 0.8438 - val_loss: 0.4231
Epoch 2/5
375/375 ----- 36s 96ms/step - accuracy: 0.8593 - loss: 0.3925 - val_accuracy: 0.8732 - val_loss: 0.3536
Epoch 3/5
375/375 ----- 31s 83ms/step - accuracy: 0.8796 - loss: 0.3269 - val_accuracy: 0.8673 - val_loss: 0.3595
Epoch 4/5
375/375 ----- 43s 115ms/step - accuracy: 0.8888 - loss: 0.3062 - val_accuracy: 0.8919 - val_loss: 0.2994
Epoch 5/5
375/375 ----- 43s 116ms/step - accuracy: 0.9012 - loss: 0.2712 - val_accuracy: 0.8987 - val_loss: 0.2832
```

```
In [7]: loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)

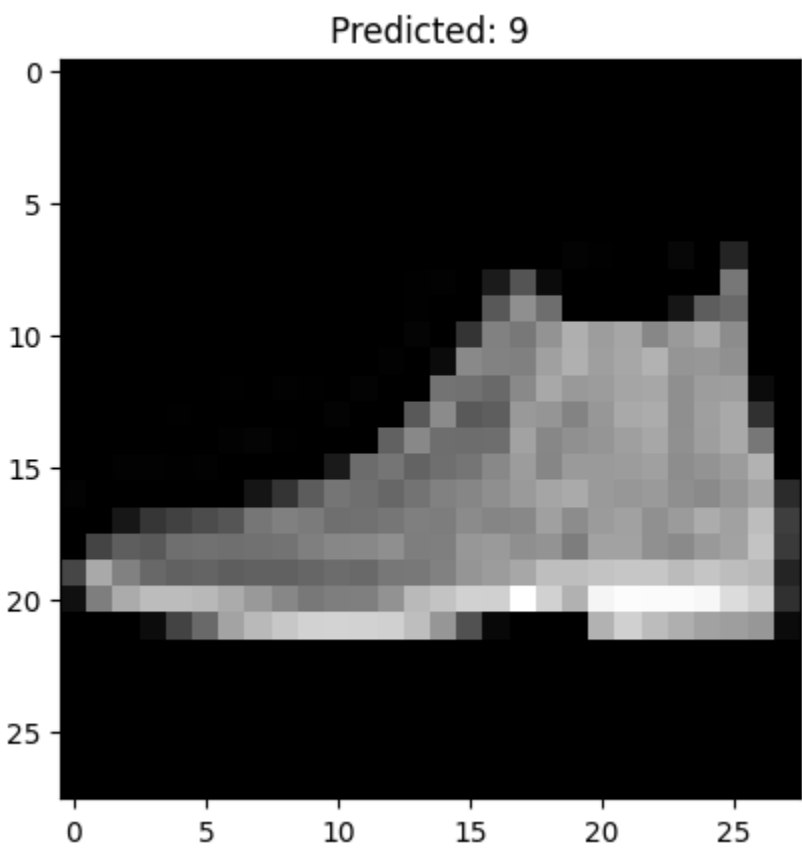
313/313 ----- 7s 24ms/step - accuracy: 0.8907 - loss: 0.3040
Test Accuracy: 0.8924000263214111
```

```
In [8]: y_pred = model.predict(X_test)

# Convert probabilities to class labels
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

313/313 ----- 4s 11ms/step
```

```
In [9]: plt.imshow(X_test[0].reshape(28,28), cmap='gray')
plt.title("Predicted: " + str(y_pred_classes[0]))
plt.show()
```



```
In [ ]:
```